Validador Sintáctico de URLs



Fundamentos Teóricos de Informática

Universidad Nacional de la Patagonia San Juan Bosco

Grupo 9

- Gonzalez Bonetto Marcos
- Cárcamo Mateo
- Gomes dos Ramos Gonzalo

1.Introducción, Objetivos Y Alcance **2**
1.1. Introducción 2
1.2. Objetivos 3
1.3. Alcance y Limitaciones del Proyecto 3
2. Fundamentación Teórica **4**
2.1. Definicion Formal del AFND 4
2.2. Teoría de Lenguajes y Estructura Gramatical 5
2.3. Expresión Regular Equivalente 7
2.4. Justificación de la Elección de un AFND sobre un AFD 11
2.5. Tabla de Transiciones del Autómata Finito No Determinista 11
Tabla de transiciones **12**
3. Gramática BNF Extendida (EBNF) **14**
4. Implementación **19**
4.1. Herramientas utilizadas 19
4.2. Arquitectura del sistema 19
4.3. Funcionamiento del validador 20
4.4. Interfaz gráfica 21
4.5. Representación del autómata 21
5. Pruebas realizadas **22**
6. Conclusiones **22**
7. Bibliografía y herramientas utilizadas **23**
Bibliografía **23**
Herramientas utilizadas **23**

1.Introducción, Objetivos Y Alcance

1.1. Introducción

La validación sintáctica de cadenas de caracteres constituye una aplicación práctica de la teoría de lenguajes formales y autómatas. En el ámbito de las aplicaciones web, las **URL (Localizadores Uniformes de Recursos)** permiten identificar recursos mediante una estructura compuesta por diferentes elementos, como el protocolo, el dominio, el puerto, la ruta y los parámetros de consulta.

El presente proyecto aborda la concepción, diseño e implementación de un sistema validador sintáctico de URLs implementado en **Python**, cuyo funcionamiento se fundamenta en un **Autómata Finito No Determinista (AFND)**. El sistema analiza una cadena ingresada por el usuario y determina si pertenece al lenguaje formal definido por las reglas del autómata.

La aplicación incorpora una interfaz gráfica desarrollada con **Tkinter**, que permite ingresar direcciones web, visualizar los resultados de validación, identificar los componentes de la URL y consultar un historial de verificaciones realizadas durante la sesión.

De esta manera, el proyecto busca relacionar los conceptos teóricos de lenguajes regulares, gramáticas y autómatas finitos con una implementación funcional.

Link del repositorio: Repositorio

1.2. Objetivos

Objetivo General

Diseñar e implementar una herramienta computacional en **Python** basada en un **Autómata Finito No Determinista (AFND)** para la validación sintáctica de URLs, capaz de reconocer sus componentes estructurales, determinar la aceptación o rechazo de las cadenas ingresadas y presentar los resultados mediante una interfaz gráfica de usuario.

1.3. Alcance y Limitaciones del Proyecto

Alcance Soportado

- **Protocolo de Red:** Reconocimiento opcional de los esquemas http:// y https://.
- **Dominio y subdominios:** reconocimiento de nombres compuestos por caracteres alfanuméricos, con posibilidad de utilizar guiones internos y múltiples niveles de subdominios.
- **Extensiones de dominio:** aceptación de extensiones de dos letras, como `.ar` y `.io`; de

tres o más letras, como **`.com`**, **`.info`** y **`.technology`**; y extensiones compuestas, como **`.com.ar`**.

- **Puerto TCP:** Identificación opcional de puertos especificados mediante : seguidos de un bloque de 1 a 5 dígitos numéricos, con una comprobación complementaria de su valor numérico dentro del rango 0 a 65535 (ej. :80, :8080).
- **Ruta de Recurso (*Path*):** Aceptación de rutas alfanuméricas de múltiples segmentos, como /productos/123, incluyendo rutas terminadas en /.
- **Parámetros de consulta (Query Strings):** reconocimiento de consultas mediante el símbolo '?', con parámetros de la forma clave=valor y múltiples parámetros separados por '&'.
- **Resultados de Validación:** Presentación del resultado de aceptación o rechazo, información sobre los componentes identificados y mensajes de diagnóstico.

Limitaciones de la Gramática Actual (Fuera del Alcance)

- No comprueba si el dominio existe, está registrado o responde a solicitudes de red.
- No verifica la seguridad, autenticidad o confiabilidad de los sitios web.
- Direcciones IP puras (IPv4 / IPv6) en sustitución del nombre de dominio.

2. Fundamentación Teórica

2.1. Definicion Formal del AFND

El núcleo del sistema de validación sintáctica de URLs se modela matemáticamente como un **Autómata Finito No Determinista (AFND)** definido mediante la quíntupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$

- ***Q:*** *conjunto finito de estados.*
- ***Σ:*** *alfabeto de símbolos de entrada.*
- ***δ:*** *función de transición.*
- ***q_0:*** *estado inicial.*
- ***F:*** *conjunto de estados finales o de aceptación.*

1. Conjunto de Estados (Q):

Q = *{ q0, q1, q2, q3, q4, q5, q6, q21, q7, subdominio, subdominio_guion, dominio_guion, 1caracter_dominio, dominio, extension, 1letra, 2letra, 3letra, tld_largo, PAIS, 1LETRA, 2LETRA, puerto, 1num, 2num, 3num, 4num, 5num, ruta, q20, consulta_inicio, consulta_clave, consulta_valor_inicio, consulta_valor}*

2. Alfabetos de Entrada (Σ):

- ***Letras (Σ_L): {a, b,..., z, A, B,...,Z}***
- ***Dígitos (Σ_D):*** $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- ***Alfanumérico (Σ_A):*** $\Sigma_L \cup \Sigma_D$
- ***Símbolos (Σ_S): { :, /, ., ?, =, -, & }***
- ***Alfabeto Total:*** $\Sigma = \Sigma_A \cup \Sigma_S$

3. Estado Inicial (q_0):

- $q_0 \in Q$

4. Conjunto de Estados de Aceptación (F):

$$F = \{3letra, 2LETRA, tld_largo, 1num, 2num, 3num, 4num, 5num, \ ruta, \ q_{20}, consulta_valor\}$$

2.2. Teoría de Lenguajes y Estructura Gramatical

El lenguaje reconocido por el autómata, L(M), pertenece a la clase de los lenguajes regulares (Tipo 3 de la Jerarquía de Chomsky). Está compuesto por la concatenación de los siguientes lenguajes formales.

Para simplificar su representación, se utilizan los alfabetos definidos en el apartado 2.1:

- ΣL: conjunto de letras.
- ΣD: conjunto de dígitos.
- ΣA: conjunto de caracteres alfanuméricos.
- λ: cadena vacía.
- *: cero o más repeticiones.
- $^+$: una o más repeticiones.

1. Lenguaje del Protocolo (L_{prot})

El protocolo es opcional y puede ser HTTP o HTTPS.

L_{prot} **= {"http://", "https://", λ}**

2. Lenguaje de las Etiquetas de Dominio ($L_{etiqueta}$**)**

Las etiquetas están formadas por caracteres alfanuméricos y admiten guiones internos, pero no al inicio ni al final.

$L_{etiqueta} = \Sigma A \cup \Sigma A \cdot (\Sigma A \cup \{"-"\})^* \cdot \Sigma A$

Ejemplos: google, mi-pagina, servidor123.

3. Lenguaje del Subdominio (L_{sub}**)**

Los subdominios son opcionales y pueden tener múltiples niveles separados por puntos.

L_{sub} **= (**$L_{etiqueta}$ **· {"."})***

Ejemplos: www., mail.google..

4. Lenguaje del Dominio Principal (L_{dom}**)**

El dominio principal está formado por una etiqueta válida.

L_{dom} **=** $L_{etiqueta}$

Ejemplos: google, ejemplo, tienda-online.

5. Lenguaje de las Extensiones (L_{TLD}**)**

Se admiten extensiones de dos letras, de tres o más letras y extensiones compuestas con un sufijo territorial de dos letras.

$L_{TLD} = \{"."\} \cdot (\Sigma L^2 \cup \Sigma L^3 \cdot \Sigma L^* \cdot (\{"."\} \cdot \Sigma L^2 \cup \{\lambda\}))$

Ejemplos: .ar, .io, .com, .technology, .com.ar.

6. Lenguaje del Puerto (L_{puerto}**)**

El puerto es opcional y está compuesto por el símbolo : seguido de uno a cinco dígitos.

L_{puerto} **= {λ} ∪ {":"} · (ΣD ∪ ΣD² ∪ ΣD³ ∪ ΣD⁴ ∪ ΣD⁵)**

Ejemplos: :80, :443, :8080.

El AFND reconoce la estructura numérica. Posteriormente, Python comprueba que el valor esté comprendido entre 0 y 65535.

7. Lenguaje de la Ruta (L_{ruta})

La ruta es opcional y puede contener múltiples segmentos alfanuméricos separados por /. También se permite una barra final.

L_{ruta} *= {λ, "/"} ∪ {"/"} · ΣA⁺ · ({"/"} · ΣA⁺)* · ({"/"} ∪ {λ})*

Ejemplos: /, /productos, /productos/123, /usuarios/perfil/.

8. Lenguaje de Consulta ($L_{consulta}$)

Los parámetros de consulta son opcionales y se componen de pares clave=valor, separados por &.

$L_{consulta}$ **= {λ} ∪ {"?"} · ΣA⁺ · {"="} · ΣA⁺ · ({"&"} · ΣA⁺ · {"="} · ΣA⁺)***

Ejemplos: ?id=10, ?pagina=2&orden=1.

Expresión Formal del Lenguaje Global

El lenguaje reconocido por el AFND se obtiene concatenando los lenguajes definidos:

$L_{URL} = L_{prot} \cdot L_{sub} \cdot L_{dom} \cdot L_{TLD} \cdot L_{puerto} \cdot L_{ruta} \cdot L_{consulta}$

Por ejemplo, la siguiente cadena pertenece al lenguaje:

https://www.mi-pagina.com.ar:8080/productos/123?pagina=2

Observación: El prefijo **www** no es obligatorio para que una URL sea aceptada. El autómata interpreta los subdominios como etiquetas alfanuméricas de longitud variable, por lo que no restringe la cantidad de letras **w** ni la cantidad de niveles de subdominio. La validez sintáctica no implica que el dominio exista o esté registrado.

2.3. Expresión Regular Equivalente

P = "http://" | "https://" | λ

E = A | A(A | "-")*A

T = LL | LLLL*("."LL | λ)

U = λ | ":"D(λ | D | DD | DDD | DDDD)

R = λ | "/" | "/"A+("/"A+)*("/" | λ)

C = λ | "?"A+"="A+("&"A+"="A+)*

Expresión regular global:

ER_URL = P · E · ("."E)* · "." · T · U · R · C

Donde:

P: Protocolo opcional.

E: etiqueta del dominio o subdominio.

T: extensión de dominio.

U: puerto opcional.

R: ruta opcional.

C: parámetros de consulta opcionales.

Gramática Regular Aceptada por el Autómata Finito

A partir del Autómata Finito No Determinista diseñado para la validación de URLs, se obtiene una gramática regular equivalente mediante la conversión AF → GR.

La gramática es de Tipo 3 según la jerarquía de Chomsky, con producciones lineales por derecha. Cada estado del autómata corresponde a un símbolo no terminal y cada transición determina una regla de producción.

1. Definición Formal de la Gramática G

La gramática se define formalmente mediante la cuádrupla:

G = (VN, VT, P, S)

Dónde:

Símbolo inicial (S):

S = q0

Conjunto de Símbolos No Terminales (VN):

VN = Q

Es decir, el conjunto de los 34 estados del AFND, definidos en el apartado 2.1.

Conjunto de Símbolos Terminales (VT):

VT = Σ = ΣL ∪ ΣD ∪ ΣS

Donde:

- ΣL = {a, b, ..., z, A, B, ..., Z}

- ΣD = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

- ΣS = { :, /, ., ?, =, -, & }

Conjunto de Reglas de Producción (P):

El conjunto P se obtiene a partir de las transiciones del AFND. Si existe una transición de un estado q hacia un estado p mediante un símbolo c, se establece la producción q → c p.

Además, para cada estado final se incluye la producción q → λ, donde λ representa la cadena vacía.

2. Conjunto de Reglas de Producción (P)

A continuación se presenta el conjunto de producciones correspondiente al AFND implementado.

Para simplificar su escritura, se emplean las siguientes abreviaturas:

- **alfanumérico:** cualquier carácter perteneciente a ΣA.

- **letra:** cualquier carácter perteneciente a ΣL.

- **numero:** cualquier carácter perteneciente a ΣD.

Estas abreviaturas representan conjuntos de caracteres, no símbolos terminales individuales.

Producciones:

q0 → h q1 | alfanumerico subdominio | alfanumerico dominio

q1 → t q2

q2 → t q3

q3 → p q4

q4 → s q5 | : q6

q5 → : q6

q6 → / q21

q21 → / q7

q7 → alfanumerico subdominio | alfanumerico dominio

subdominio → alfanumerico subdominio | - subdominio_guion | . 1caracter_dominio

subdominio_guion → - subdominio_guion | alfanumerico subdominio

1caracter_dominio → alfanumerico dominio

dominio → alfanumerico dominio | - dominio_guion | . extension | . 1caracter_dominio | . PAIS

dominio_guion → - dominio_guion | alfanumerico dominio

extension → letra 1letra

1letra → letra 2letra

2letra → letra 3letra

3letra → letra tld_largo | . PAIS | : puerto | / ruta | ? consulta_inicio | λ

tld_largo → letra tld_largo | . PAIS | : puerto | / ruta | ? consulta_inicio | λ

PAIS → letra 1LETRA

1LETRA → letra 2LETRA

2LETRA → : puerto | / ruta | ? consulta_inicio | λ

puerto → numero 1num

1num → numero 2num | / ruta | ? consulta_inicio | λ

2num → numero 3num | / ruta | ? consulta_inicio | λ

3num → numero 4num | / ruta | ? consulta_inicio | λ

4num → numero 5num | / ruta | ? consulta_inicio | λ

5num → / ruta | ? consulta_inicio | λ

ruta → alfanumerico q20 | ? consulta_inicio | λ

q20 → alfanumerico q20 | / ruta | ? consulta_inicio | λ

consulta_inicio → alfanumerico consulta_clave

consulta_clave → alfanumerico consulta_clave | = consulta_valor_inicio

consulta_valor_inicio → alfanumerico consulta_valor

consulta_valor → alfanumerico consulta_valor | & consulta_inicio | λ

2.4. Justificación de la Elección de un AFND sobre un AFD

Para el desarrollo del validador sintáctico de URLs se seleccionó un Autómata Finito No Determinista (AFND), debido a la flexibilidad que ofrece para representar las diferentes estructuras que pueden aparecer durante el reconocimiento de una dirección web.

La elección se fundamenta en los siguientes aspectos:

1. Ambigüedad inicial del protocolo

Una URL puede comenzar con http://, https:// o directamente con un nombre de dominio. Por ejemplo, el carácter h puede corresponder al inicio del protocolo HTTP o al primer carácter de un dominio como hola.com.

El AFND permite representar ambas posibilidades mediante transiciones simultáneas desde el estado inicial q0.

2. Reconocimiento de dominios y extensiones

Durante la lectura de un dominio, el símbolo . puede indicar la aparición de otro subdominio, el comienzo de una extensión de dos letras o el comienzo de una extensión de tres o más letras.

Por ejemplo, las cadenas www.google.com, google.ar y google.com.ar requieren interpretar los puntos según la estructura que continúa.

El AFND permite explorar estas alternativas mediante múltiples transiciones desde el estado dominio, manteniendo las posibilidades de reconocimiento hasta determinar si alguna alcanza un estado final.

3. Representación modular de la URL

El autómata organiza el reconocimiento mediante grupos de estados asociados al protocolo, dominio, extensiones, puerto, rutas y parámetros de consulta.

Esta organización facilita la interpretación de las transiciones y la incorporación de nuevas funcionalidades.

Si bien un AFD puede reconocer el mismo lenguaje regular, el modelo no determinista resultó conveniente para representar directamente las distintas alternativas sintácticas del proyecto.

2.5. Tabla de Transiciones del Autómata Finito No Determinista

La función de transición δ determina los posibles estados siguientes según el estado actual y el símbolo leído.

A diferencia de un Autómata Finito Determinista, el AFND permite que una combinación de estado y símbolo produzca un conjunto de estados posibles.

Para representar las transiciones de manera más compacta se utilizan las siguientes abreviaturas:

- **A:** cualquier carácter alfanumérico (ΣA).
- **L:** cualquier letra (ΣL).
- **D:** cualquier dígito (ΣD).
- **A − {h}:** cualquier carácter alfanumérico excepto la letra h.

- ∅**:** ausencia de transición.
- ***:** identifica un estado final o de aceptación.

Tabla de transiciones

Estado actual	Símbolo → Estado(s) siguiente(s)
→ q0	h → {q1, subdominio, dominio}; A − {h} → {subdominio, dominio}
q1	t → {q2}
q2	t → {q3}
q3	p → {q4}
q4	s → {q5}; : → {q6}
q5	: → {q6}
q6	/ → {q21}
q21	/ → {q7}
q7	A → {subdominio, dominio}
subdominio	A → {subdominio}; - → {subdominio_guion}; . → {1caracter_dominio}
subdominio_guion	A → {subdominio}; - → {subdominio_guion}
1caracter_dominio	A → {dominio}
dominio	A → {dominio}; - → {dominio_guion}; . → {extension, 1caracter_dominio, PAIS}
dominio_guion	A → {dominio}; - → {dominio_guion}
extension	L → {1letra}

1letra	L → {2letra}
2letra	L → {3letra}
*3letra	L → {tld_largo}; . → {PAIS}; : → {puerto}; / → {ruta}; ? → {consulta_inicio}
*tld_largo	L → {tld_largo}; . → {PAIS}; : → {puerto}; / → {ruta}; ? → {consulta_inicio}
PAIS	L → {1LETRA}
1LETRA	L → {2LETRA}
*2LETRA	: → {puerto}; / → {ruta}; ? → {consulta_inicio}
puerto	D → {1num}
*1num	D → {2num}; / → {ruta}; ? → {consulta_inicio}
*2num	D → {3num}; / → {ruta}; ? → {consulta_inicio}
*3num	D → {4num}; / → {ruta}; ? → {consulta_inicio}
*4num	D → {5num}; / → {ruta}; ? → {consulta_inicio}
*5num	/ → {ruta}; ? → {consulta_inicio}
*ruta	A → {q20}; ? → {consulta_inicio}
*q20	A → {q20}; / → {ruta}; ? → {consulta_inicio}
consulta_inicio	A → {consulta_clave}
consulta_clave	A → {consulta_clave}; = → {consulta_valor_inicio}
consulta_valor_inicio	A → {consulta_valor}

*consulta_valor	A → {consulta_valor}; & → {consulta_inicio}

Las combinaciones de estado y símbolo que no aparecen en la tabla corresponden a transiciones no definidas, cuyo resultado es el conjunto vacío (∅).

El autómata reconoce una cadena cuando, después de consumir todos sus caracteres, al menos uno de los estados activos pertenece al conjunto de estados finales F.

Ejemplo de no determinismo

Para ilustrar el funcionamiento del AFND, se considera el estado dominio al recibir el símbolo .:

δ(dominio, ".") = {extension, 1caracter_dominio, PAIS}

Esto significa que el autómata contempla simultáneamente tres posibles interpretaciones:

- extension: comienzo de una extensión de tres o más letras.
- 1caracter_dominio: comienzo de otra etiqueta del dominio.
- PAIS: comienzo de una extensión de dos letras.

El reconocimiento continúa por las alternativas posibles y la cadena se acepta si alguna de ellas termina en un estado final después de consumir toda la entrada.

Esta característica permite representar distintas estructuras de URL sin tener que seleccionar anticipadamente un único recorrido.

3. Gramática BNF Extendida (EBNF)

La estructura sintáctica de las URLs reconocidas por el Autómata Finito No Determinista puede representarse mediante la notación EBNF (Extended Backus–Naur Form), una extensión de BNF que permite describir de manera compacta elementos opcionales y estructuras repetitivas.

Esta representación establece las reglas necesarias para construir las cadenas aceptadas por el AFND, contemplando protocolos, dominios, extensiones, puertos, rutas y parámetros de consulta.

Para interpretar las producciones se utilizan las siguientes convenciones:

- `< >`: delimitan símbolos no terminales.
- `::=`: indica una regla de producción.
- `|`: representa alternativas.

- `[ ]`: indica que un elemento es opcional.
- `{ }`: indica cero o más repeticiones.
- `" "`: identifica símbolos terminales.

El símbolo inicial de la gramática es `<Url>`.

3.1. Símbolos Terminales (VT)

El conjunto de símbolos terminales está formado por:

- Letras minúsculas: `a-z`.
- Letras mayúsculas: `A-Z`.
- Dígitos: `0-9`.
- Símbolos especiales: `:`, `/`, `.`, `?`, `=`, `-`, `&`.

El alfabeto corresponde al conjunto de símbolos definidos para el AFND en el apartado 2.1.

3.2. Reglas de Producción EBNF

1. Estructura general de la URL

```
<Url> ::= [ <Esquema> ] <Dominio>
        [ <Puerto> ] [ <Ruta> ] [ <Consulta> ]

<Esquema> ::= "http://" | "https://"
```

La URL debe contener un dominio válido, mientras que el protocolo, el puerto, la ruta y los parámetros de consulta son opcionales.

2. Dominio y subdominios

```
<Dominio> ::= <Etiqueta> { "." <Etiqueta> } <Extension>

<Etiqueta> ::= <Alfanumerico>
            { <Alfanumerico> | <SecuenciaGuion> }

<SecuenciaGuion> ::= "-" { "-" } <Alfanumerico>
```

Estas producciones permiten dominios y subdominios de longitud variable, compuestos por caracteres alfanuméricos y guiones internos.

Los guiones no pueden aparecer al inicio ni al final de una etiqueta. Asimismo, el prefijo `www` no es obligatorio y no se limita la cantidad de niveles de subdominio.

3. Extensiones de Dominio (TLD)

```
<Extension> ::= "." <TLD>
```

```
<TLD> ::= <Letra> <Letra>
        | <Letra> <Letra> <Letra>
          { <Letra> }
          [ "." <Letra> <Letra> ]
```

La extensión debe contener al menos dos letras.

Se reconocen extensiones de dos letras, como `.ar` y `.io`; de tres o más letras, como `.com`, `.info` y `.technology`; y terminaciones compuestas, como `.com.ar`.

4. Puerto

```
<Puerto> ::= ":" <Digito>
             [ <Digito>
               [ <Digito>
                 [ <Digito>
                   [ <Digito> ]
                 ]
               ]
             ]
```

El puerto puede estar formado por uno a cinco dígitos.

El AFND reconoce su estructura sintáctica, mientras que la aplicación verifica posteriormente que el valor numérico esté comprendido entre 0 y 65535.

5. Ruta de Recurso (Path)

```
<Ruta> ::= "/" [ <Segmento>
                 { "/" <Segmento> }
                 [ "/" ]
               ]

<Segmento> ::= <Alfanumerico> { <Alfanumerico> }
```

Estas producciones permiten rutas simples, múltiples segmentos y barras finales.

Por ejemplo:

- `/`
- `/productos`
- `/productos/123`
- `/usuarios/perfil/`

No se admiten segmentos vacíos intermedios, como en `/productos//123`.

6. Parámetros de Consulta (Query Strings)

```
<Consulta> ::= "?" <Parametro>
               { "&" <Parametro> }

<Parametro> ::= <Clave> "=" <Valor>

<Clave> ::= <Alfanumerico> { <Alfanumerico> }

<Valor> ::= <Alfanumerico> { <Alfanumerico> }
```

Las consultas se representan mediante pares `clave=valor`, separados por el símbolo `&`.

Cada clave y cada valor deben contener al menos un carácter alfanumérico.

Por ejemplo:

- `?id=10`
- `?pagina=2&orden=1`

7. Definición de Caracteres

```
<Alfanumerico> ::= <Letra> | <Digito>

<Letra> ::= <Minuscula> | <Mayuscula>

<Minuscula> ::= "a" | "b" | "c" | "d" | "e" | "f"
              | "g" | "h" | "i" | "j" | "k" | "l"
              | "m" | "n" | "o" | "p" | "q" | "r"
              | "s" | "t" | "u" | "v" | "w" | "x"
              | "y" | "z"

<Mayuscula> ::= "A" | "B" | "C" | "D" | "E" | "F"
              | "G" | "H" | "I" | "J" | "K" | "L"
              | "M" | "N" | "O" | "P" | "Q" | "R"
              | "S" | "T" | "U" | "V" | "W" | "X"
              | "Y" | "Z"

<Digito> ::= "0" | "1" | "2" | "3" | "4"
           | "5" | "6" | "7" | "8" | "9"
```

3.3. Ejemplo de Aplicación de la Gramática

Se considera la siguiente URL:

`https://www.mi-pagina.com.ar:8080/productos/123?id=10&orden=2`

De acuerdo con las producciones EBNF, la cadena se descompone en los siguientes componentes:

Componente	Cadena Reconocida
Esquema	https://
Dominio	www.mi-pagina.com.ar
Puerto	:8080
Ruta	/productos/123
Consulta	?id=10&orden=2

El dominio contiene dos etiquetas (`www` y `mi-pagina`) y una extensión compuesta (`.com.ar`).

La ruta contiene dos segmentos alfanuméricos, mientras que la consulta presenta dos parámetros separados mediante `&`.

Todos los componentes cumplen las producciones definidas, por lo que la cadena pertenece al lenguaje reconocido por el AFND y supera la comprobación numérica del puerto.

3.4. Correspondencia entre la EBNF y el AFND

La gramática EBNF describe la estructura de las cadenas que el AFND reconoce mediante estados y transiciones.

Ambas representaciones corresponden al mismo lenguaje regular: la gramática establece cómo pueden construirse las cadenas, mientras que el autómata determina si una cadena ingresada pertenece a dicho lenguaje.

La validación numérica del puerto se realiza de manera complementaria mediante Python y no forma parte de las restricciones expresadas por la gramática.

Por lo tanto, la EBNF representa el lenguaje sintáctico reconocido por el autómata, mientras que la aplicación incorpora las comprobaciones adicionales necesarias para determinar el resultado final de validación.

4. Implementación

El proyecto fue desarrollado en Python y consiste en una aplicación de escritorio destinada a verificar la estructura sintáctica de direcciones URL mediante un Autómata Finito No Determinista (AFND).
Para facilitar el desarrollo, el mantenimiento y la organización del sistema, se separó la lógica de validación de la interfaz gráfica, utilizando dos módulos principales.

4.1. Herramientas utilizadas

Para el desarrollo se emplearon las siguientes herramientas:

- **Python**: lenguaje de programación utilizado para implementar el autómata y la aplicación.
- **automata-lib**: biblioteca que permite definir los estados, el alfabeto, las transiciones y los estados de aceptación del AFND.
- **Tkinter**: biblioteca estándar de Python utilizada para desarrollar la interfaz gráfica de escritorio.
- **urllib.parse:** módulo estándar empleado para identificar y mostrar los componentes de las URLs que superan la validación.
- **Graphviz**: herramienta utilizada de forma auxiliar para generar una representación gráfica del autómata y sus transiciones.
- **JFLAP:** herramienta utilizada para representar y analizar el modelo formal del autómata.
-Pillow (PIL): biblioteca utilizada por el visor del AFND para mostrar, redimensionar y ampliar la imagen del grafo.

4.2. Arquitectura del sistema

La aplicación está organizada en tres archivos principales:

detector.py — Motor de validación

Contiene la definición formal del AFND, sus 34 estados, el alfabeto de entrada, la función de transición y los estados finales.

También implementa las funciones encargadas de procesar las cadenas ingresadas, determinar los estados activos durante el reconocimiento y comprobar si se alcanza algún estado de aceptación.

Adicionalmente, incorpora la comprobación numérica del puerto y funciones auxiliares para reconstruir recorridos aceptados y generar el grafo del autómata.

validador.py — Interfaz gráfica

Implementa la interfaz de escritorio mediante Tkinter y permite que el usuario ingrese una dirección URL para su validación.

Este módulo se comunica con el motor de validación y presenta el resultado obtenido, indicando si la cadena cumple o no con el formato admitido.

Cuando la URL es aceptada, también muestra sus componentes principales: protocolo, dominio, puerto, ruta y parámetros de consulta.

La separación entre módulos permite modificar la presentación de la aplicación sin alterar las reglas formales del autómata.

visor_grafo.py — Visor del autómata: módulo auxiliar que permite abrir el diagrama del AFND en una ventana independiente, utilizar herramientas de zoom y desplazamiento, y actualizar la imagen a partir de las transiciones implementadas.

4.3. Funcionamiento del validador

El proceso de validación se desarrolla mediante las siguientes etapas:

1. Ingreso de la URL

El usuario introduce una dirección en el campo de texto de la aplicación y solicita su verificación mediante el botón correspondiente o la tecla Enter.

2. Preparación de la cadena

El programa elimina los espacios en blanco ubicados al comienzo y al final de la entrada. También normaliza el protocolo HTTP o HTTPS cuando está escrito con letras mayúsculas o minúsculas.

3. Procesamiento mediante el AFND

El motor inicia el reconocimiento en el estado `q0` y procesa la cadena carácter por carácter.

Para cada símbolo leído, calcula el conjunto de estados alcanzables desde los estados activos. Debido al no determinismo, durante la lectura pueden existir varios estados activos simultáneamente.

4. Comprobación de aceptación

Una vez consumida toda la cadena, se verifica si al menos uno de los estados alcanzados pertenece al conjunto de estados finales del AFND.

Si no se alcanza ningún estado final, la cadena es rechazada.

5. Validación complementaria del puerto

Si la cadena es aceptada por el autómata y contiene un puerto, el programa comprueba que su valor numérico se encuentre entre 0 y 65535.

Por ejemplo, `https://ejemplo.com:8080` supera esta comprobación, mientras que `https://ejemplo.com:65536` es rechazada.

6. Presentación del resultado

Finalmente, la interfaz informa si la URL cumple con el formato admitido. Cuando es aceptada, presenta sus componentes; cuando es rechazada, muestra un mensaje de diagnóstico.

4.4. Interfaz gráfica

La interfaz fue diseñada para permitir una validación sencilla de direcciones web, sin necesidad de que el usuario conozca el funcionamiento interno del autómata.

Sus principales funcionalidades son:

- Campo de entrada para escribir una URL.
- Botón para ejecutar la validación.
- Ejemplos predefinidos para realizar comprobaciones.
- Indicador visual de aceptación o rechazo.
- Visualización de los componentes de la URL aceptada.
- Mensajes que informan posibles errores de formato.
- Historial de las últimas cuatro verificaciones realizadas durante la sesión.

La aplicación permite consultar el grafo del AFND mediante un visor independiente, sin interferir con la interfaz principal de validación. Esta funcionalidad facilita la relación entre el reconocimiento sintáctico de las URLs y el modelo formal desarrollado, permitiendo visualizar sus estados y transiciones.

De esta manera, el usuario interactúa con una herramienta de validación, mientras que el procesamiento de las cadenas se realiza mediante el modelo formal desarrollado.

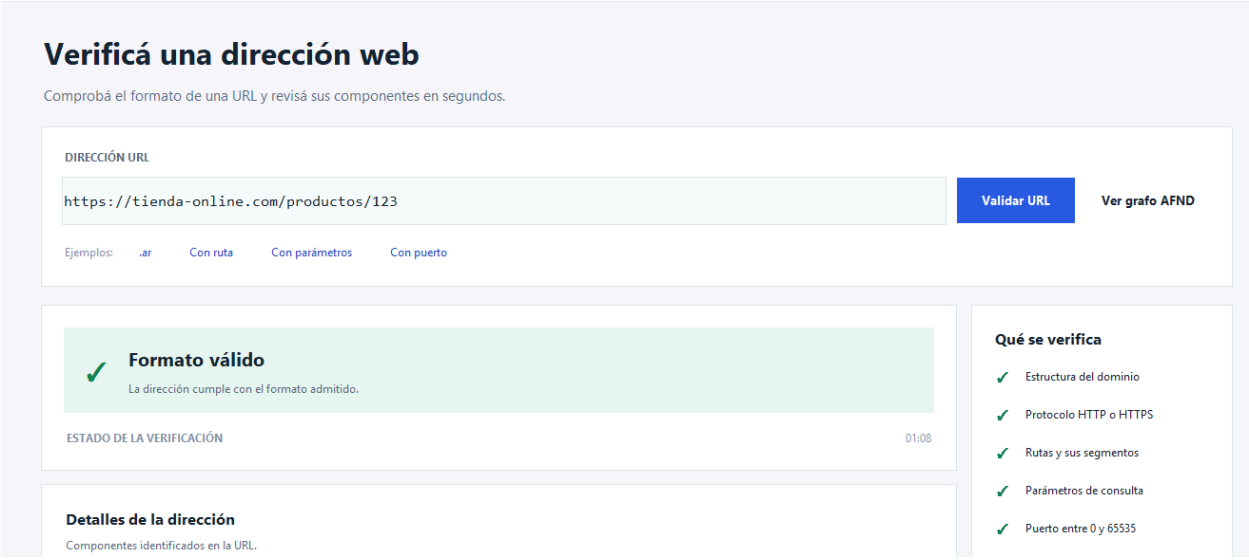


Figura 1. Interfaz principal del programa durante la validación sintáctica de una dirección URL. Fuente: elaboración propia.

4.5. Representación del autómata

El modelo formal del sistema se encuentra representado mediante un diagrama de estados que permite visualizar sus transiciones, el estado inicial y los estados de aceptación.

Para generar esta representación se utiliza Graphviz, a partir de las transiciones definidas en el motor de validación. También se dispone de una representación del autómata en formato JFLAP para su análisis.

Estas herramientas permiten relacionar el comportamiento de la aplicación con los fundamentos teóricos de los autómatas finitos no deterministas.

El visor también permite actualizar la representación gráfica a partir de las transiciones definidas en el motor de validación, manteniendo la correspondencia entre el modelo implementado y su diagrama de estados.

5. Pruebas realizadas

Palabra probada	Resultado esperado	Resultado obtenido	Motivo
google.com	Aceptada	Aceptada	Dominio con extensión válida
https://ejemplo.com.ar	Aceptada	Aceptada	Incluye protocolo y código de país
www.mail.google.com	Aceptada	Aceptada	Múltiples subdominios
https://google.technology	Aceptada	Aceptada	Extensión larga
https://mi-pagina.com	Aceptada	Aceptada	Guión interno
https://google.com/productos/123/	Aceptada	Aceptada	Ruta múltiple y barra final
https://google.com?x=1&y=2	Aceptada	Aceptada	Parámetros de consulta
https://google.com:65535	Aceptada	Aceptada	Puerto en el límite permitido
https://google.com:65536	Rechazada	Rechazada	Puerto fuera de rango

https://pagina-.com	Rechazada	Rechazada	Guion mal ubicado
https://google.com?x=	Rechazada	Rechazada	Parámetro sin valor

6. Conclusiones

El desarrollo nos permitió aplicar los conceptos de lenguajes formales y autómatas finitos en una herramienta funcional destinada a la validación sintáctica de direcciones URL. Mediante la implementación de un Autómata Finito No Determinista (AFND), se logró representar y reconocer distintas estructuras, incluyendo protocolos, dominios, subdominios, extensiones, puertos, rutas y parámetros de consulta.

La utilización del AFND permitió modelar diferentes alternativas de reconocimiento mediante estados y transiciones, mientras que su representación mediante gramáticas regulares, expresiones regulares y notación EBNF facilitó la relación entre el modelo matemático y su implementación en Python. Asimismo, la separación entre el motor de validación y la interfaz gráfica permitió organizar el sistema y facilitar su mantenimiento.

Si bien el validador presenta limitaciones respecto de los estándares completos de URL y no comprueba la existencia ni la seguridad de los sitios web, cumple con el objetivo de reconocer las estructuras sintácticas definidas para el proyecto. La experiencia permitió comprender de manera práctica cómo los fundamentos teóricos de la informática pueden utilizarse para resolver problemas concretos de reconocimiento y validación de cadenas.

7. Bibliografía y herramientas utilizadas

Para el desarrollo del proyecto se utilizaron materiales de referencia sobre lenguajes formales y autómatas finitos, así como herramientas de programación, representación gráfica y validación.

Bibliografía

- **Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2007).** *Introduction to Automata Theory, Languages, and Computation* (3.ª ed.). Pearson. Referencia para los conceptos de autómatas finitos, lenguajes regulares, gramáticas y expresiones regulares.

  (https://drive.google.com/drive/folders/1JpEYcm3lUfCJvKHao59logYpPN2_Wj0K?usp=s

haring)

Herramientas utilizadas

- **Python:** lenguaje de programación empleado para desarrollar el sistema.
- **automata-lib:** biblioteca utilizada para implementar el modelo formal del AFND.
- **Tkinter:** biblioteca estándar de Python utilizada para desarrollar la interfaz gráfica.
- **urllib.parse:** módulo estándar de Python empleado para descomponer las URLs en sus componentes.
- **Graphviz:** herramienta utilizada para generar el diagrama de estados y transiciones del autómata.
- **JFLAP:** herramienta utilizada para representar y analizar el autómata finito.
- **GitHub:** plataforma utilizada para almacenar y gestionar el repositorio del proyecto.
- **ChatGPT (OpenAI):** herramienta de apoyo utilizada para consultas conceptuales, revisión de documentación y asistencia durante el desarrollo y la organización del informe.